What is RNN?

A recurrent neural network (RNN) is a deep learning model that is trained to process and convert a sequential data input into a specific sequential data output. Sequential data is data—such as words, sentences, or time-series data—where sequential components interrelate based on complex semantics and syntax rules. An RNN is a software system that consists of many interconnected components mimicking how humans perform sequential data conversions, such as translating text from one language to another. RNNs are largely being replaced by transformer-based artificial intelligence (AI) and large language models (LLM), which are much more efficient in sequential data processing.

How does a recurrent neural network work?

The following image shows a diagram of an RNN.

RNNs are made of neurons: data-processing nodes that work together to perform complex tasks. The neurons are organized as input, output, and hidden layers. The input layer receives the information to process, and the output layer provides the result. Data processing, analysis, and prediction take place in the hidden layer.

Hidden layer

RNNs work by passing the sequential data that they receive to the hidden layers one step at a time. However, they also have a self-looping or *recurrent* workflow: the hidden layer can remember and use previous inputs for future predictions in a short-term memory component. It uses the current input and the stored memory to predict the next sequence.

For example, consider the sequence: *Apple is red.* You want the RNN to predict *red* when it receives the input sequence *Apple is.* When the hidden layer processes the word *Apple*, it stores a copy in its memory. Next, when it sees the word *is*, it recalls *Apple* from its memory and understands the full sequence: *Apple is* for context. It can then predict *red* for improved accuracy. This makes RNNs useful in speech recognition, machine translation, and other language modeling tasks.

Training

Machine learning (ML) engineers train deep neural networks like RNNs by feeding the model with training data and refining its performance. In ML, the neuron's weights are signals to determine how influential the information learned during training is when predicting the output. Each layer in an RNN shares the same weight.

ML engineers adjust weights to improve prediction accuracy. They use a technique called backpropagation through time (BPTT) to calculate model error and adjust its weight accordingly. BPTT rolls back the output to the previous time step and recalculates the error rate. This way, it can identify which hidden state in the sequence is causing a significant error and readjust the weight to reduce the error margin.

What are the types of recurrent neural networks?

RNNs are often characterized by one-to-one architecture: one input sequence is associated with one output. However, you can flexibly adjust them into various configurations for specific purposes. The following are several common RNN types.

One-to-many

This RNN type channels one input to several outputs. It enables linguistic applications like image captioning by generating a sentence from a single keyword.

Many-to-many

The model uses multiple inputs to predict multiple outputs. For example, you can create a language translator with an RNN, which analyzes a sentence and correctly structures the words in a different language.

Many-to-one

Several inputs are mapped to an output. This is helpful in applications like sentiment analysis, where the model predicts customers' sentiments like *positive, negative*, and *neutral* from input testimonials.

How do recurrent neural networks compare to other deep learning networks?

RNNs are one of several different neural network architectures.

Recurrent neural network vs. feed-forward neural network

Like RNNs, feed-forward neural networks are artificial neural networks that pass information from one end to the other end of the architecture. A feed-forward neural network can perform simple classification, regression, or recognition tasks, but it can't remember the previous input that it has processed. For example, it forgets *Apple* by the time its neuron processes the word *is*. The RNN overcomes this memory limitation by including a hidden memory state in the neuron.

Recurrent neural network vs. convolutional neural networks

Convolutional neural networks are artificial neural networks that are designed to process spatial data. You can use convolutional neural networks to extract spatial information from videos and images by passing them through a series of convolutional and pooling layers in the neural network. RNNs are designed to capture long-term dependencies in sequential data

What are some variants of recurrent neural network architecture?

The RNN architecture laid the foundation for ML models to have language processing capabilities. Several variants have emerged that share its memory retention principle and improve on its original functionality. The following are some examples.

Bidirectional recurrent neural networks

A bidirectional recurrent neural network (BRNN) processes data sequences with forward and backward layers of hidden nodes. The forward layer works similarly to the RNN, which stores the previous input in the hidden state and uses it to predict the subsequent output. Meanwhile, the backward layer works in the opposite direction by taking both the current input and the future hidden state to update the present hidden state. Combining both layers enables the BRNN to improve prediction accuracy by considering past and future contexts. For example, you can use the BRNN to predict the word *trees* in the sentence *Apple trees are tall*.

Long short-term memory

Long short-term memory (LSTM) is an RNN variant that enables the model to expand its memory capacity to accommodate a longer timeline. An RNN can only remember the immediate past input. It can't use inputs from several previous sequences to improve its prediction.

Consider the following sentences: *Tom is a cat. Tom's favorite food is fish*. When you're using an RNN, the model can't remember that Tom is a cat. It might generate various foods when it predicts the last word. LSTM networks add a special memory block called *cells* in the hidden layer. Each cell is controlled by an input gate, output gate, and forget gate, which enables the layer to remember helpful information. For example, the cell remembers the words *Tom* and *cat*, enabling the model to predict the word *fish*.

Gated recurrent units

A gated recurrent unit (GRU) is an RNN that enables selective memory retention. The model adds an update and forgets the gate to its hidden layer, which can store or remove information in the memory.

What are the limitations of recurrent neural networks?

Since the RNN's introduction, ML engineers have made significant progress in natural language processing (NLP) applications with RNNs and their variants. However, the RNN model family has several limitations.

Read about natural language processing

Exploding gradient

An RNN can wrongly predict the output in the initial training. You need several iterations to adjust the model's parameters to reduce the error rate. You can describe the sensitivity of the error rate corresponding to the model's parameter as a gradient. You can imagine a gradient as a slope that you take to descend from a hill. A steeper gradient enables the model to learn faster, and a shallow gradient decreases the learning rate.

Exploding gradient happens when the gradient increases exponentially until the RNN becomes unstable. When gradients become infinitely large, the RNN behaves erratically, resulting in performance issues such as overfitting. Overfitting is a phenomenon where the model can predict accurately with training data but can't do the same with real-world data.

Vanishing gradient

The vanishing gradient problem is a condition where the model's gradient approaches zero in training. When the gradient vanishes, the RNN fails to learn effectively from the training data, resulting in underfitting. An underfit model can't perform well in real-life applications because its weights weren't adjusted appropriately. RNNs are at risk of vanishing and exploding gradient issues when they process long data sequences.

Slow training time

An RNN processes data sequentially, which limits its ability to process a large number of texts efficiently. For example, an RNN model can analyze a buyer's sentiment from a couple of sentences. However, it requires massive computing power, memory space, and time to summarize a page of an essay.

How do transformers overcome the limitations of recurrent neural networks?

Transformers are deep learning models that use self-attention mechanisms in an encoder-decoder feed-forward neural network. They can process sequential data the same way that RNNs do.

Self-attention

Transformers don’t use hidden states to capture the interdependencies of data sequences. Instead, they use a self-attention head to process data sequences in parallel. This enables transformers to train and process longer sequences in less time than an RNN does. With the self-attention mechanism, transformers overcome the memory limitations and sequence interdependencies that RNNs face. Transformers can process data sequences in parallel and use positional encoding to remember how each input relates to others.

Parallelism

Transformers solve the gradient issues that RNNs face by enabling parallelism during training. By processing all input sequences simultaneously, a transformer isn’t subjected to backpropagation restrictions because gradients can flow freely to all weights. They are also optimized for parallel computing, which graphic processing units (GPUs) offer for generative AI developments. Parallelism enables transformers to scale massively and handle complex NLP tasks by building larger models.